Компьютерный практикум

Тема 8. Сортировки

Преподаватель: Д. В. Давыдов

2026

Insertion Sort: пошаговое выполнение

а) i = 2, ключ = 2

←	-	-	-	-	-
5	2	4	6	1	3

Insertion Sort: пошаговое выполнение

а) i = 2, ключ = 2

←	-	-	-	-	-
5	2	4	6	1	3

б) i = 3, ключ = 4

|	←	-	-	-	-
2	5	4	6	1	3

Insertion Sort: пошаговое выполнение

а) i = 2, ключ = 2

←	-	-	-	-	-
5	2	4	6	1	3

б) i = 3, ключ = 4

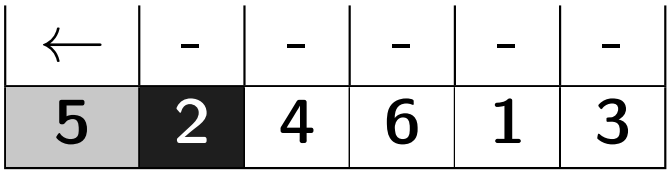
|	←	-	-	-	-
2	5	4	6	1	3

в) i = 4, ключ = 6

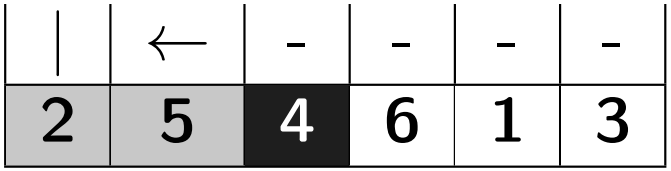
|	|	|	-	-	-
2	4	5	6	1	3

Insertion Sort: пошаговое выполнение

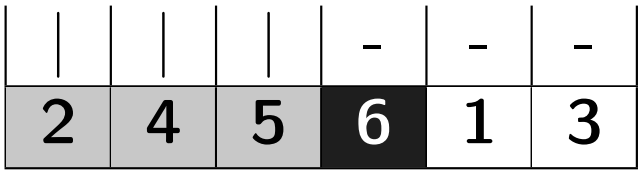
а) i = 2, ключ = 2



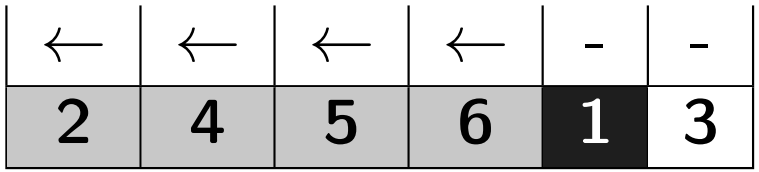
б) i = 3, ключ = 4



в) i = 4, ключ = 6

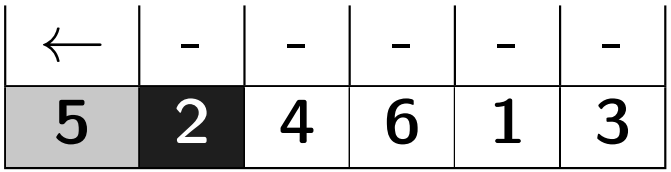


г) i = 5, ключ = 1

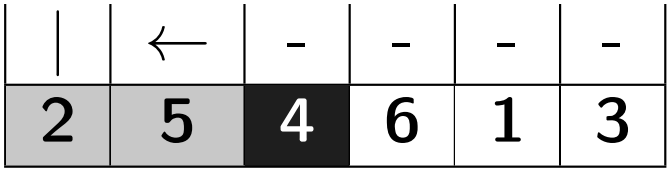


Insertion Sort: пошаговое выполнение

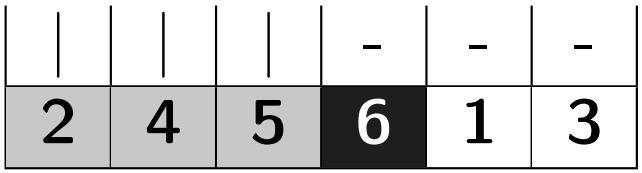
а) i = 2, ключ = 2



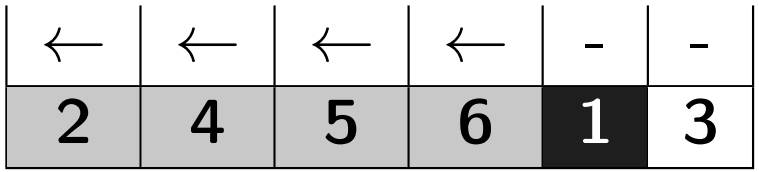
б) i = 3, ключ = 4



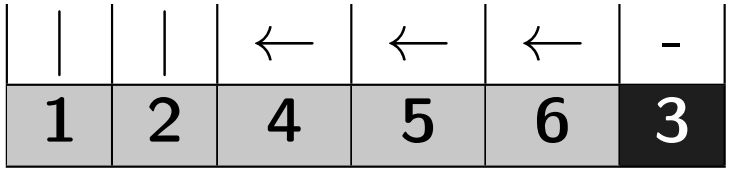
в) i = 4, ключ = 6



г) i = 5, ключ = 1

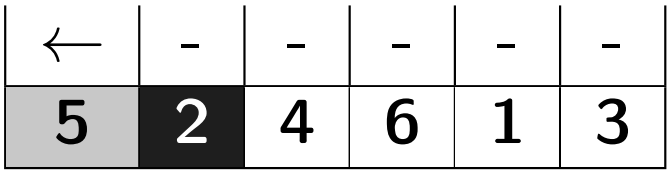


д) i = 6, ключ = 3

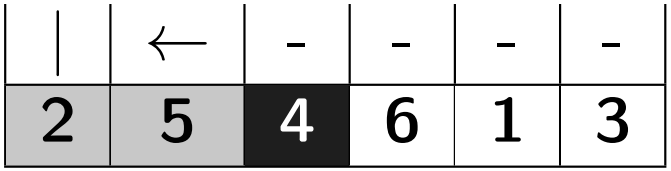


Insertion Sort: пошаговое выполнение

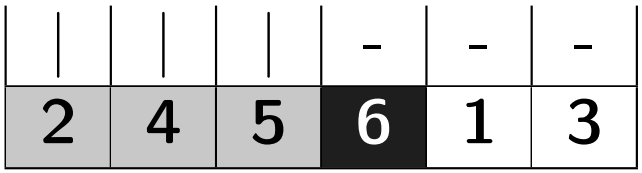
а) i = 2, ключ = 2



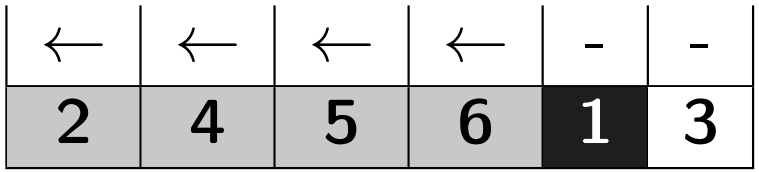
б) i = 3, ключ = 4



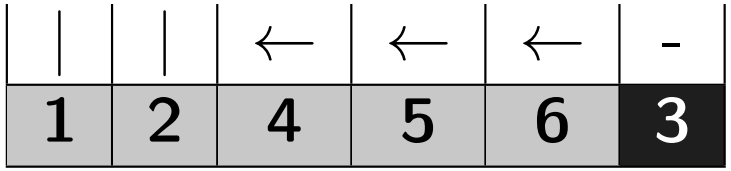
в) i = 4, ключ = 6



г) i = 5, ключ = 1



д) i = 6, ключ = 3



е) Результат



Insertion sort псевдокод

INSERTION_SORT(A)

```
1   for j ← 2 to length[A]
2     do key ← A[j]
3        ▷ Вставка элемента A[j] в отсортированную
4        ▷ последовательность A[1 .. j − 1]
5        i ← j − 1
6        while i > 0 и A[i] > key
7          do A[i + 1] ← A[i]
8             i ← i − 1
9        A[i + 1] ← key
```

Insertion sort время работы

Говорят, что в худшем случай (изначально массив отсортирован по убыванию) время работы алгоритма: $T(n) = \Theta(n^2)$, где n — размер исходного массива.
А в лучшем (массив отсортирован по возрастанию) — $\Theta(n)$. см. [Кормен, гл. 2]

Метод декомпозиции

Подход "разделяй и властвуй". Рекурсивная структура решения заключается в 3 этапах:

- **Разделение** задачи на несколько подзадач
- **Покорение** — рекурсивное решение подзадач
- **Комбинирование** решения задачи из решений подзадач

Merge sort

- **Разделение** массива пополам
- **Покорение** — сортировка последовательностей Merge sort
- **Комбинирование** — слияние (Merge) двух отсортированных последовательностей.

Функция MERGE(A, p, q, r)

MERGE(A, p, q, r)

```
1  n1 ← q − p + 1
2  n2 ← r − q
3  new L[1 .. n1 + 1] и R[1 .. n2 + 1]
4  for i ← 1 to n1
5     do L[i] ← A[p + i − 1]
6  for j ← 1 to n2
7     do R[j] ← A[q + j]
8  L[n1 + 1] ← ∞
9  R[n2 + 1] ← ∞
10 i ← 1
11 j ← 1
12 for k ← p to r
13    do if L[i] ⩽ R[j]
14       then A[k] ← L[i]
15            i ← i + 1
16    else A[k] ← R[j]
17       j ← j + 1
```

Функция MERGE_SORT(A, p, r)

MERGE_SORT(A, p, r)

```
1    if p < r
2       then q ← ⌊(p + r)/2⌋
3          MERGE_SORT(A, p, q)
4          MERGE_SORT(A, q + 1, r)
5          MERGE(A, p, q, r)
```

Ещё про сортировки

Всё, что было выше в общем виде называется **сортировки сравнением**. Для произвольного набора данных в массиве подобные алгоритмы не могут быть быстрее $\Theta(n \ln n)$ в худшем случае.
Однако, если мы обладаем дополнительной информацией, то можно написать сортировку быстрее.

Задача: Пусть для массива A произвольной длины n, его значения являются цифрами ($A[i] \in \{0, 1, \ldots, 9\} \quad \forall i \in [0, N]$).
Требуется отсортировать массив.

Примечание к заданию

Требуется создать дополнительный массив B длины 10 (количество цифр) значение которого для i-ого индекса будет то, сколько раз встречается цифра i в массиве A.